

FIT3182 Asg3: Cloud-Deployable Stream Analytics

Operationalizing Distributed Pipelines via AWS Free Tier Architecture

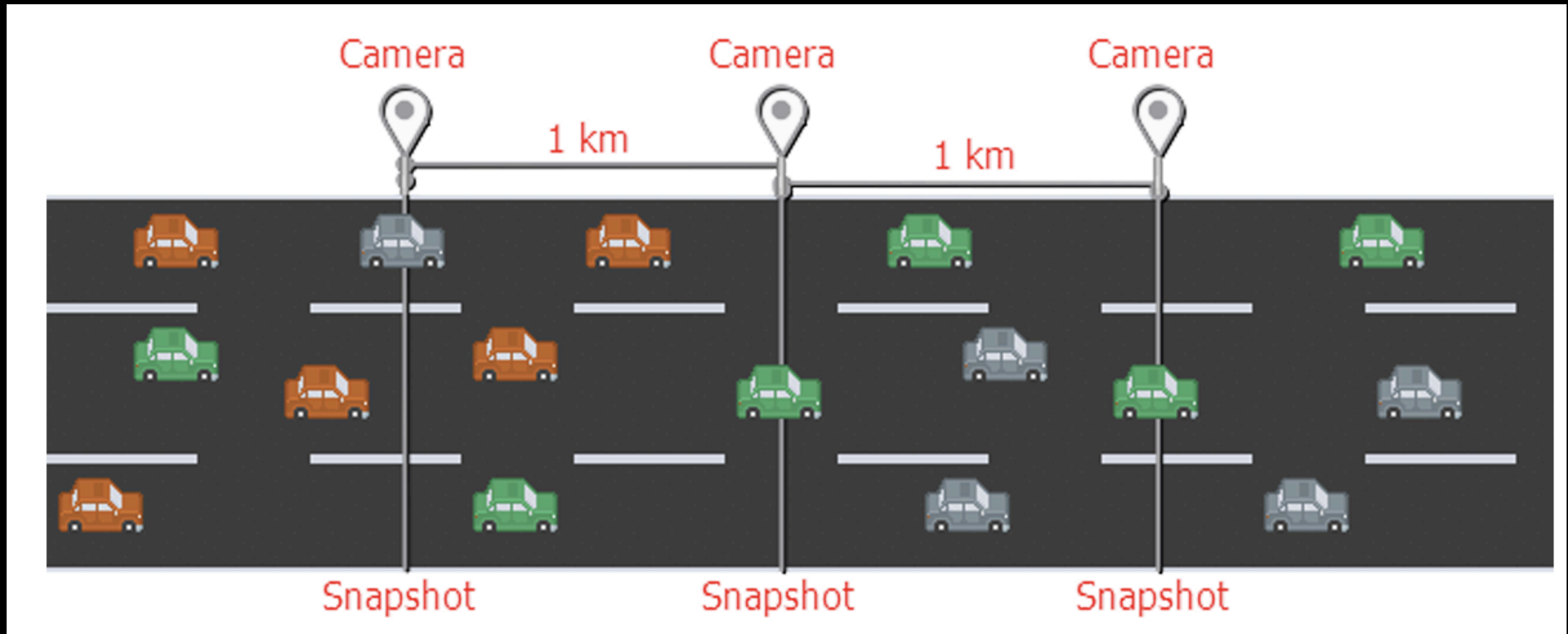
From Prototype to Production Cloud

Transitioning Assignment 2 to a Decoupled, Robust AWS Infrastructure

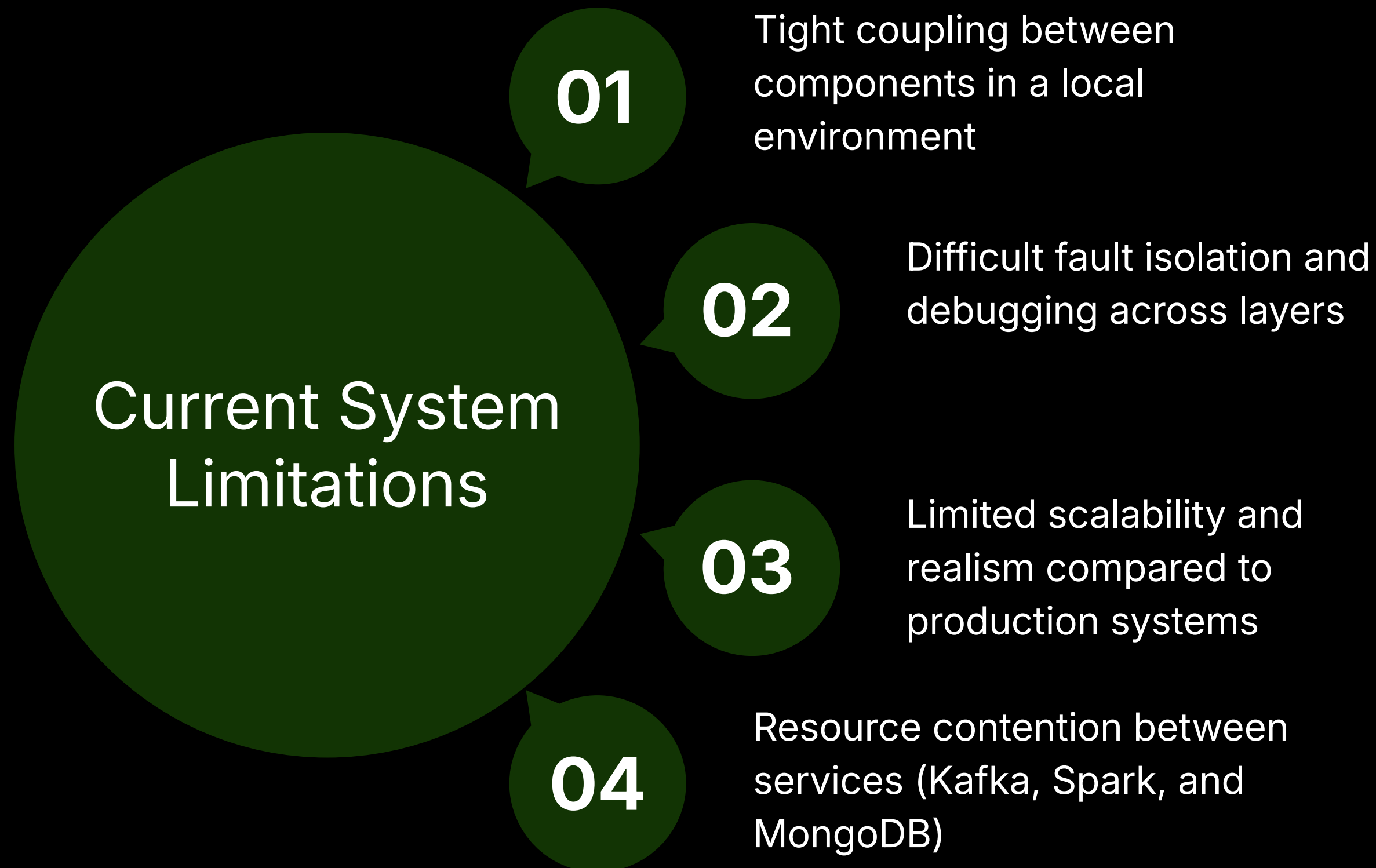
Team member : Ngu Khang Wei (34838260)
Garent Ngor (35123656)

Generative AI Declaration: We used chatgpt for guidance on how to setup aws instances, coming up with ideas, debugging issues and error

Quick Recap



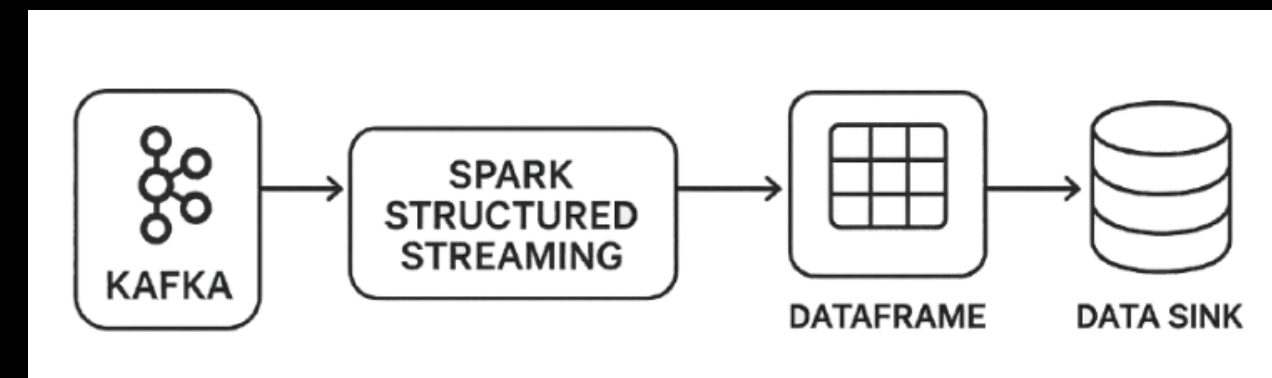
Problem Statement



Literature Foundation

Kafka + Spark Streaming Architecture (Kelvin, 2023)

- Kafka used as a high-throughput ingestion layer
- Spark Structured Streaming used for real-time processing
- Demonstrated:
 - lower latency
 - higher throughput
 - scalable streaming pipeline design
- Key Insight
 - Decoupling ingestion and processing improves scalability and fault tolerance



https://www.researchgate.net/publication/396645280_INTEGRATING_APACHE_KAFKA_WITH_SPARK_STRUCTURED_STREAMING_FOR_LOW-LATENCY_DATA_PROCESSING_IN_BIG_DATA_ARCHITECTURES

KAFKA TUNING & CONFIGURATION (Hung et al., 2025)

Implemented pipeline:

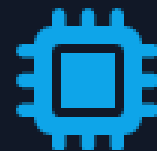
Kafka → Spark Structured Streaming → MongoDB Atlas

kafka Configuration



3 Topics

Dedicated streams for camera-events-A, B, and C to maintain logical separation.



3 Partitions

Enabled parallel processing across topics to optimize Spark throughput.



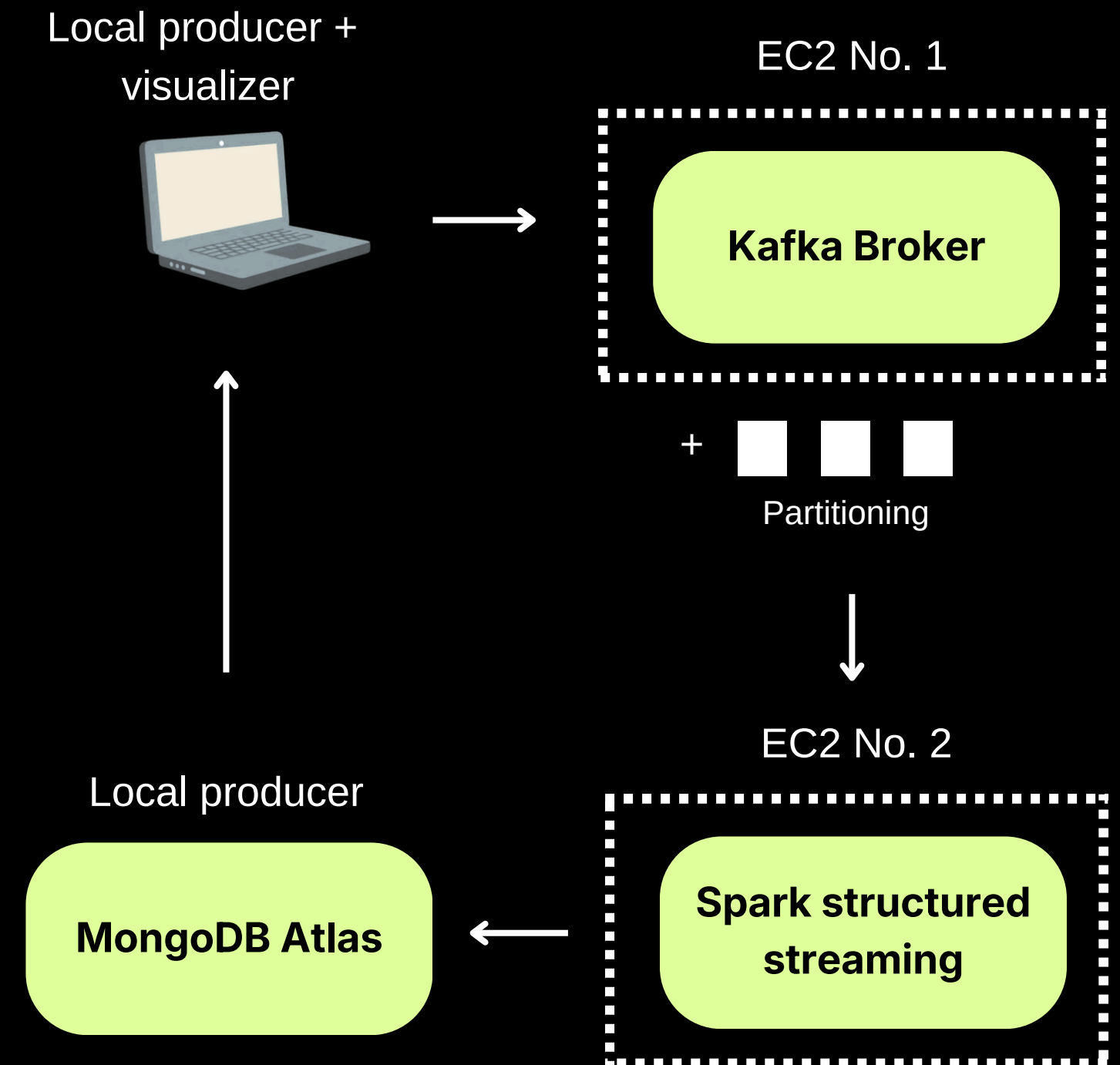
Replica Factor 1

Optimized for Free Tier constraints while maintaining decoupling benefits.

System Architecture & Decoupling

We implemented a strictly decoupled architecture to ensure fault tolerance and resource efficiency:

-  **Local Producers:** Laptop offloads generation load from cloud CPUs.
-  **AWS Kafka Node:** Managed message brokering and persistence.
-  **AWS Spark Node:** Isolated processing engine with memory tuning.
-  **MongoDB Atlas:** Global sink layer for high-availability storage.



Benefits of Chosen Architecture

- Loose Coupling Between Components
- Improved Scalability Through Partitioning
- Real-Time Stream Processing Capability
- Better Fault Isolation and Debugging
- Cloud-Based Flexibility (AWS EC2 + MongoDB Atlas)
- Adapted Efficiency Under Resource Constraints
- High Throughput Data Pipeline

Innovation: Infrastructure as Code



Automated Setup

Bash scripts (setup-kafka.sh) automate OS updates, Docker installs, and dynamic IP mapping to ensure 100% reproducibility.



Containerization

Docker-Compose orchestrates Zookeeper and Kafka clusters with specific memory limits to fit AWS Free Tier boundaries.



Dynamic Routing

Scripts query AWS metadata to resolve Public IPs automatically, eliminating manual hardcoding during reboots.

Technical Trade-offs & Limitations

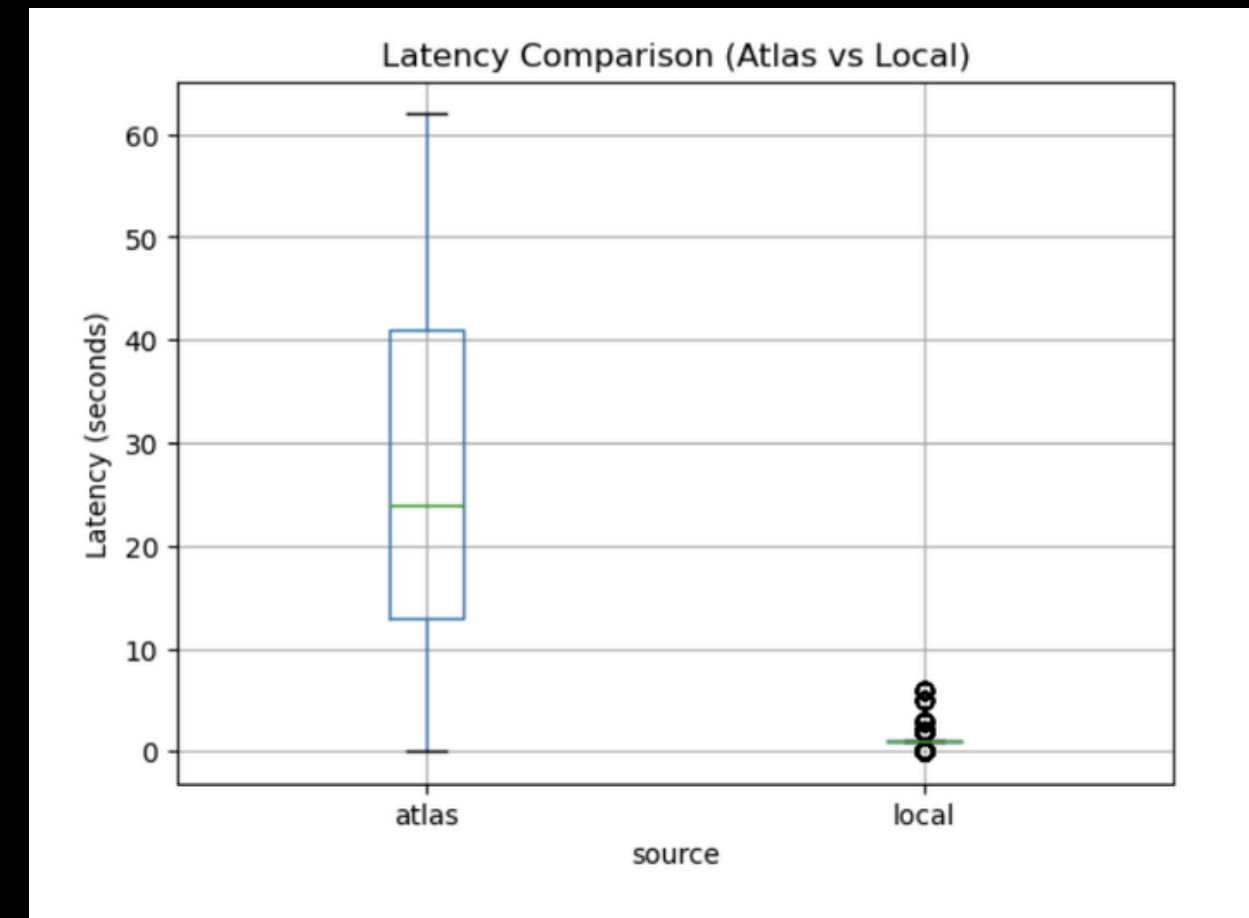
Analyzing the balance between Cost, Performance, and Complexity

Latency Analysis

```
{  
  "batch_id": 1,  
  "camera_id": 101,  
  "producer_id": "producer_A",  
  "events": [  
    {  
      "event_id": "e1",  
      "car_plate": "ABC123",  
      "timestamp": "2026-06-12T15:30:00Z",  
      "speed_reading": "72.5",  
      "producer_sent_time": "2026-06-12T15:29:58Z"  
    }  
  ]  
}
```



*latency = ingest_time -
producer_sent_time*

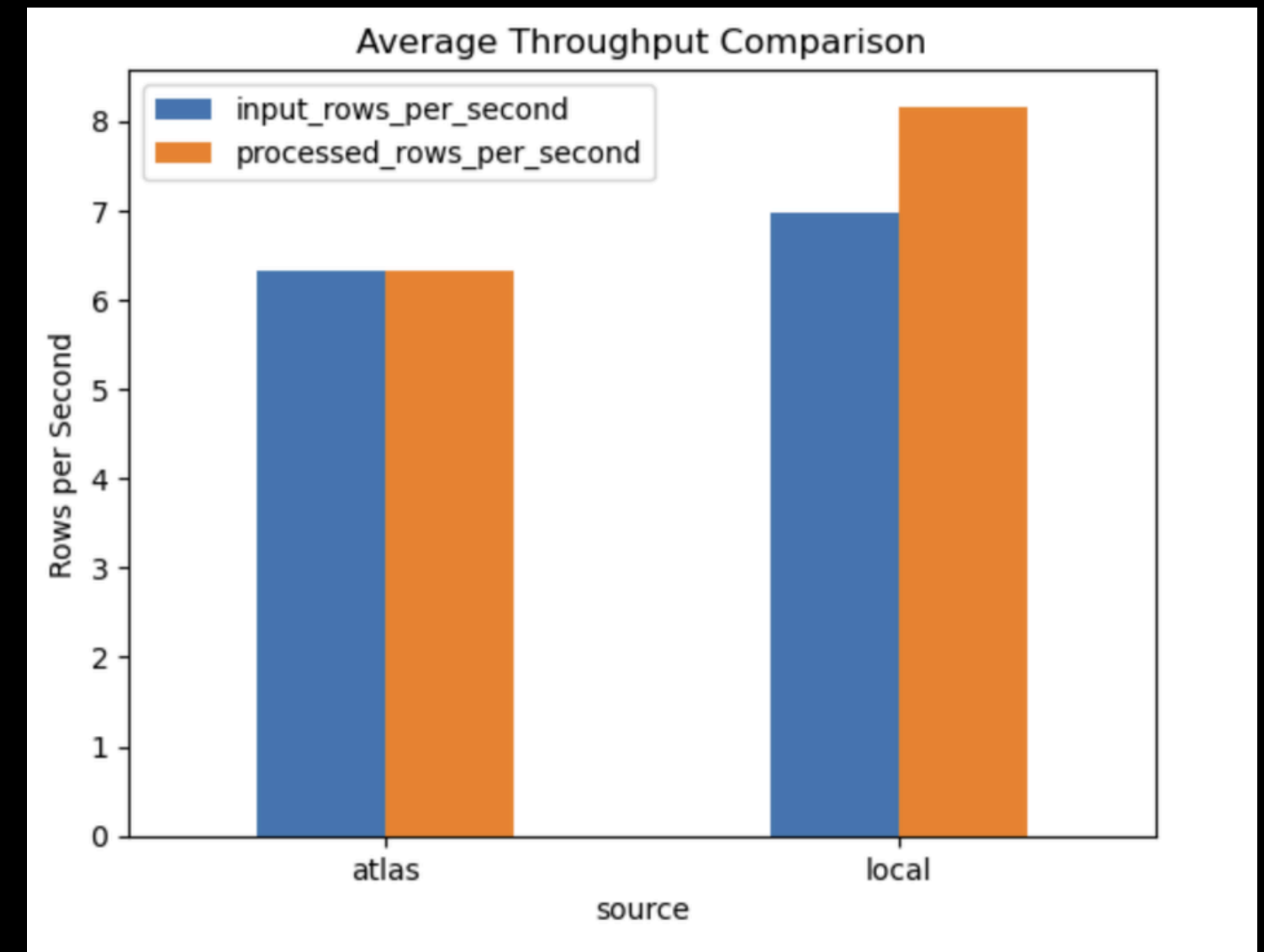


Throughput Analysis

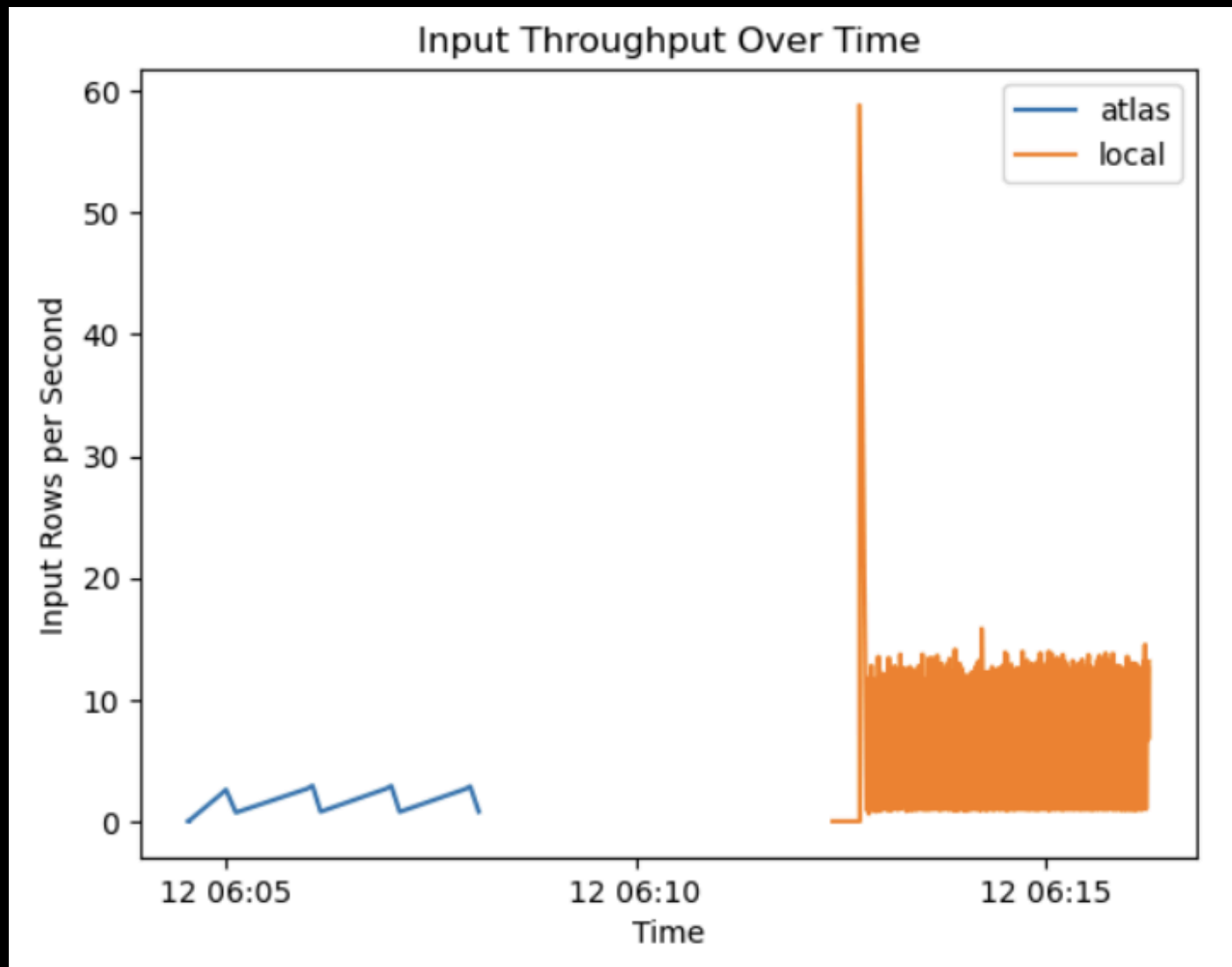
- 🔄 Real-time Monitoring Loop
 - 1 second background monitoring loop
 - query.lastProgress API to extract batch-level metrics
 - Checks query.isActive to monitor running queries
 - New micro-batch is recorded using batchId tracking

Analysis:

- Slightly higher input rows per second and processed rows per second for local version
- Limited RAM and storage on EC2 → Cloud version is performing relatively well



One caveat



- Although local deployment may achieve higher input throughput, the cloud-based architecture provides improved consistency and more stable system - performance under varying workloads.

Future improvement

- Full Distributed Cluster Deployment (Kafka & Spark Clusters for Higher Scalability)
 - Auto-Scaling with Managed Cloud Services (AWS EKS / MSK / EMR)
 - Real-Time Monitoring & Observability (Prometheus + Grafana Dashboarding)
 - Data Schema Management using Schema Registry (Avro / Protobuf for Data Consistency)
-

Research Articles / Blogs

- https://www.researchgate.net/publication/396645280_INTEGRATING_APACHE_KAFKA_WITH_SPARK_STRUCTURED_STREAMING_FOR_LOW-LATENCY_DATA_PROCESSING_IN_BIG_DATA_ARCHITECTURES
- <https://helda.helsinki.fi/server/api/core/bitstreams/876b8c1d-5266-4e37-95b4-472811fd1edc/content>

Other references

https://www.oreilly.com/radar/questioning-the-lambda-architecture/?_gl=1*1jeekdj*_gcl_au*MTc2MTUxMzc5OC4xNzg5MTc3NDMx

<https://streamkap.com/resources-and-guides/kappa-architecture-guide>

<https://ieeexplore.ieee.org/document/9466838>

<https://lsds.doc.ic.ac.uk/sites/default/files/p361-theodorakis.pdf>

https://www.researchgate.net/publication/354173084_A_COMPARATIVE_STUDY_ON_DISTRIBUTED_COMPUTING_AND_CLOUD_COMPUTING

“ Verma & Mehta (2021): Kafka acts as a distributed backbone, decoupling ingestion rate from processing capacity. Justifies our EC2 separation.

“ Sharma & Li (2020): Micro-batch tuning significantly reduces end-to-end latency while maintaining Exactly-Once Semantics.

“ Confluent Architecture: Aligns Kafka keys with MongoDB `\_id` to implement Idempotent Upserts, ensuring data integrity during retries.

Code demo

The end

Thank you for your attention.

FIT3182: Big Data Management & Processing | Monash University